Chapter 1: The core idea — prediction, training, and feedback

If you have ever noticed your smartphone finishing your sentences as you type, you have already encountered the fundamental mechanism behind the most powerful AI systems in the world. Large Language Models (LLMs) like ChatGPT, Claude, and Gemini are, at their core, sophisticated autocomplete engines. Understanding this single insight is key to using these tools wisely — and to knowing when to trust them and when not to.

1.1 Sophisticated autocomplete: how LLMs actually work

Every time you interact with an LLM, the model is doing one thing: predicting the most likely next word[1] given everything that came before it. The model reads your prompt and then, word by word, guesses what should come next. It repeats this process until it has produced a full response.

Consider a simple example: 'Twinkle, twinkle, little ___.' Nearly every English speaker — and every LLM — will predict that the next word is 'star.'

But now try: 'The cat sat on the ___.' There are multiple plausible continuations: mat, floor, sofa, roof. The model has trained on trillions of words drawn from scanned books or scraped from the internet and it knows how frequently each possible word occurs. It then randomly selects the next word.

This is why these systems are sometimes called 'stochastic[2] parrots'. They produce fluent, contextually appropriate text by parroting statistical patterns, not by understanding meaning.

This has profound practical consequences. The most important one is that what you get from an LLM may not be correct — these models can randomly select words that generate results that are factually incorrect. This is referred to as a 'hallucination'. More on that in the next chapter.

---

[1] LLMs actually analyze and predict tokens. A token can be a whole word, part of a word (subword), a single character, or punctuation like "," or "?".

[2] The word 'stochastic' means random or probabilistic. In this context, it means that every word the model produces is the result of a weighted random draw.

In addition, because they give you plausible, fluent answers to questions you ask, you can anthropomorphize them and come to believe these chatbots are sentient. They are not.

1.2 Probability distributions: the engine of output

When an LLM generates a response, it does not simply pick the single most likely word every time. Instead, it produces a probability distribution over a vast vocabulary — assigning a likelihood to every possible next word. It then samples from that distribution, meaning lower-probability words still get picked sometimes.

To make this concrete, consider a simple example. Suppose the model has just read the phrase 'The cat sat on the …' and must predict the next word. It does not know the 'right' answer — instead, it has learned from trillions of words of text that certain words tend to follow this phrase more often than others. It assigns a probability to every word in its vocabulary, then randomly picks one. The table below shows what that might look like for this phrase:

Table 1.1. Illustrative next-word probabilities for the phrase 'The cat sat on the …'.

Possible Next Word	Probability
mat	42%
floor	28%
sofa	18%
roof	8%
table	4%

The model will pick 'mat' 42% of the time, but it will also pick less probable words. It will pick 'floor' 28% of the time, 'sofa' 18% of the time, and so on. Let's assume that it picks the word 'floor'.

Now the model repeats the same process. The sentence is now: 'The cat sat on the floor …'. The probability distribution of the next word changes. Words like 'and,' 'near,' or 'beside' may now become more likely, while others drop off. For example, the distribution might look like this:

Table 1.2. Illustrative next-word probabilities after the model has chosen 'floor'.

Possible Next Word	Probability
and	35%

Possible Next Word	Probability
near	25%
beside	18%
, (comma)	12%
. (period)	6%
looking	4%

The model again samples from that distribution. The process continues iteratively. Each new token updates the context, and that updated context reshapes the probability distribution for the next token. In other words, if the model had picked a word different from 'mat', the possible next words and their probabilities would be different. This randomness is why asking the same question twice will frequently yield two different answers.

Nevertheless, this process produces text that appears coherent because each word choice is statistically aligned with patterns learned from large amounts of data. There is no global plan formed at the start. Coherence emerges from these sequential predictions.

1.3 Key mental model: the very good guesser

Perhaps the most important thing to internalize from this chapter is a single mental model: an LLM is a very good guesser, not a logical thinker. It has processed trillions of words and learned patterns about how language works — which words follow which, how professional writing is structured, common rhetorical constructs ('it's not X, it's Y'). But it does not reason from first principles. It does not consult facts. It guesses, based on patterns.

This mental model explains a great deal. It explains why LLMs produce plausible answers that are sometimes correct. When it does, it seems like magic. But it also explains why they can be confidently incorrect.

None of this means LLMs are useless — far from it. They are extraordinarily powerful tools for drafting, editing, summarizing, translating, and coding assistance. But they are tools that must be operated with understanding.

How I want you to think about LLMs: Think of yourself not as a writer using an LLM, but as an editor. You supply the facts, the direction, the ideas — and you verify and correct whatever the LLM produces.

1.4 How LLMs are trained

Modern Large Language Models are the product of a multi-stage training pipeline involving vast quantities of data, enormous computational resources, and human judgment. Understanding how LLMs are trained helps explain both their impressive abilities and their significant limitations — including why they sometimes confidently state incorrect information.

Pre-training: learning from the internet

The first and most fundamental stage of LLM development is called pre-training. During pre-training, a model is exposed to an enormous corpus of text — trillions of words drawn from websites, books, academic papers, Wikipedia, code repositories, and more. The model's training objective during this phase is deceptively simple: given a fragment of text, find a list of plausible next words and the probability that each will show up.

By learning to predict text at massive scale, the model implicitly learns grammar, factual associations, reasoning patterns, coding syntax, conversational conventions, and much more. It learns that 'The capital of France is __' is almost always followed by 'Paris.'

It learns that Python function definitions start with the keyword 'def'. It learns that a certain style of rhetoric tends to accompany certain political arguments. None of this was explicitly programmed — it was all absorbed from the statistical patterns in the training data.

The data used in pre-training is critically important and often controversial. Datasets like Common Crawl (a massive snapshot of the web) form the backbone of most training corpora. This means the model learns from an imperfect, often biased representation of human knowledge — whatever was on the internet. Misinformation, prejudice, and cultural skew in that data all get absorbed into the model, a problem we will explore later. They also trained on copyrighted material like books and newspapers. The authors and publishers were not given any opportunity to opt out of, or otherwise control, how their works are used.

Fine-tuning: shaping the model into an assistant

A freshly pre-trained model is not very useful as a conversational assistant. It can predict text brilliantly, but if you prompt it with a question, it might respond by generating more questions in the same style, because that is what appeared in its training data. To convert a raw language model into a helpful assistant, developers use a process called Supervised Fine-Tuning (SFT).

In SFT, the model is trained on a carefully curated dataset of prompt-response pairs written or approved by humans. Examples might include: a user asks 'How do I write a cover letter?' and a skilled writer provides an ideal answer. The model learns to recognize this as the expected format for interactions and begins to produce responses that look like helpful assistant outputs rather than raw internet text. Fine-tuning requires far less compute than pre-training and can be done with tens of thousands of examples rather than the trillions of words needed for the initial training.

Fine-tuning is also how companies customize general-purpose models for specific domains. A hospital might fine-tune a model on medical literature and clinical guidelines so it responds appropriately to healthcare questions. A law firm might fine-tune on legal documents and case law.

Reinforcement learning: teaching the model what humans prefer

SFT has a limitation: it can teach the model to mimic good responses, but it cannot easily teach subtler qualities like helpfulness, appropriate tone, or safety. For these, researchers developed Reinforcement Learning from Human Feedback (RLHF), a technique that has become central to the development of modern assistants like ChatGPT and Claude.

RLHF works in three steps. First, the model generates multiple possible responses to a given prompt. Second, human raters — called 'annotators' — compare the responses and rank them from best to worst according to guidelines provided by the company. These rankings reflect qualities like accuracy, helpfulness, safety, and tone. This is then used to fine-tune the model. Through this process, the LLM learns to produce the kinds of responses humans prefer.

The result is a model that has been shaped to produce outputs that match human preferences, not just outputs that are statistically likely given internet text. This is what makes chatbots like ChatGPT feel helpful and conversational rather than like a raw text firehose. It's also why the models tend to be sycophantic, something we'll talk about more later.

1.5 Key takeaways

- LLMs work by predicting the most probable next word given all preceding text — they are advanced autocomplete, not reasoning engines.
- Outputs are selected via weighted random sampling from a probability distribution, making LLMs non-deterministic: the same prompt can yield different results each time.
- LLMs have no built-in fact-checker. They produce plausible-sounding text, which is not the same as accurate text.
- The right mental model: treat an LLM as a skilled but unreliable assistant. You are the editor who verifies and corrects.

1.6 Further reading

- Be nice to your chatbot! Some research suggests that LLMs often perform better when you encourage them.